

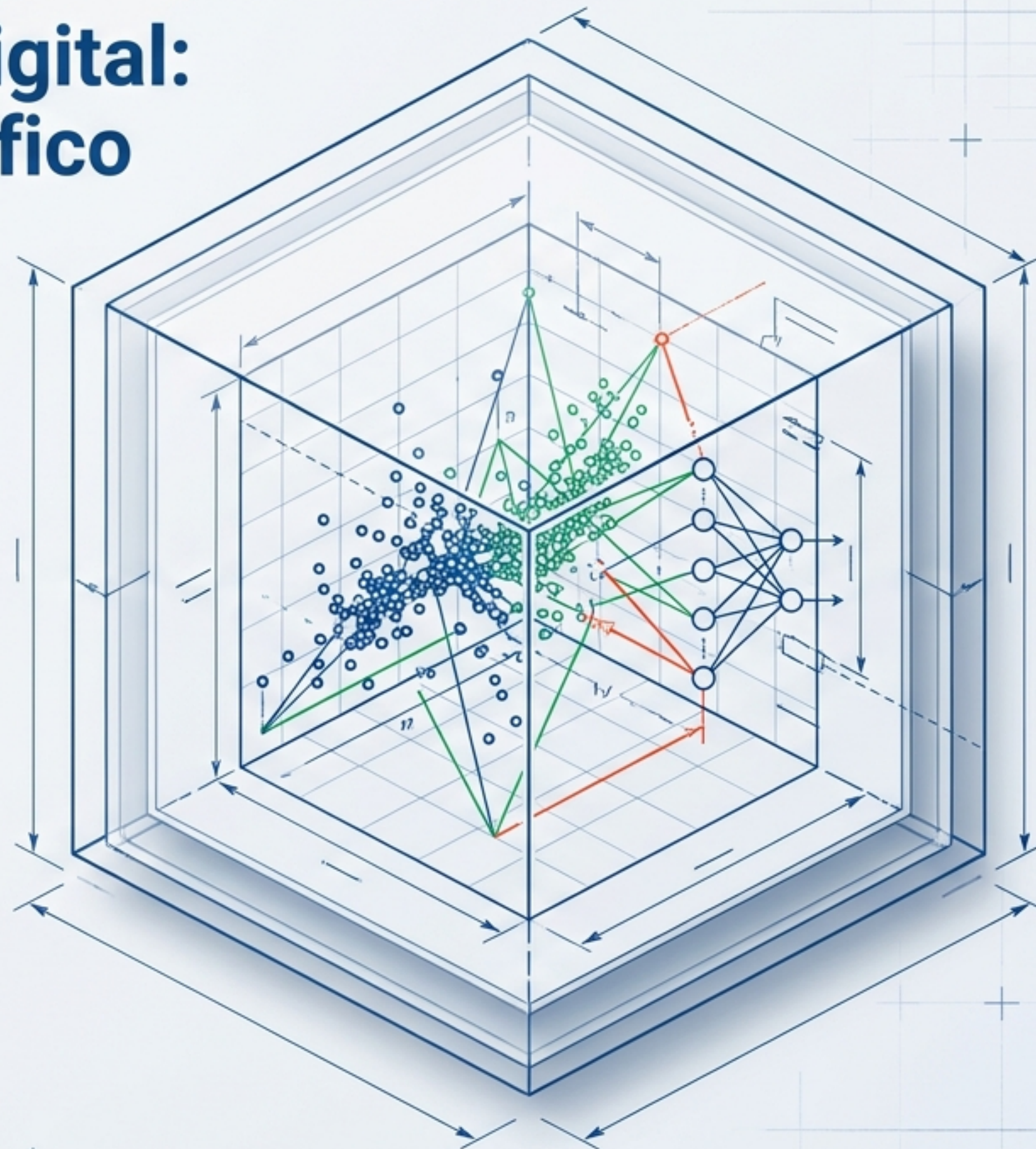
El Entorno de Laboratorio Digital: Docker para Software Científico

De la experimentación local a la reproducibilidad global: Contenedores, dependencias y despliegue.

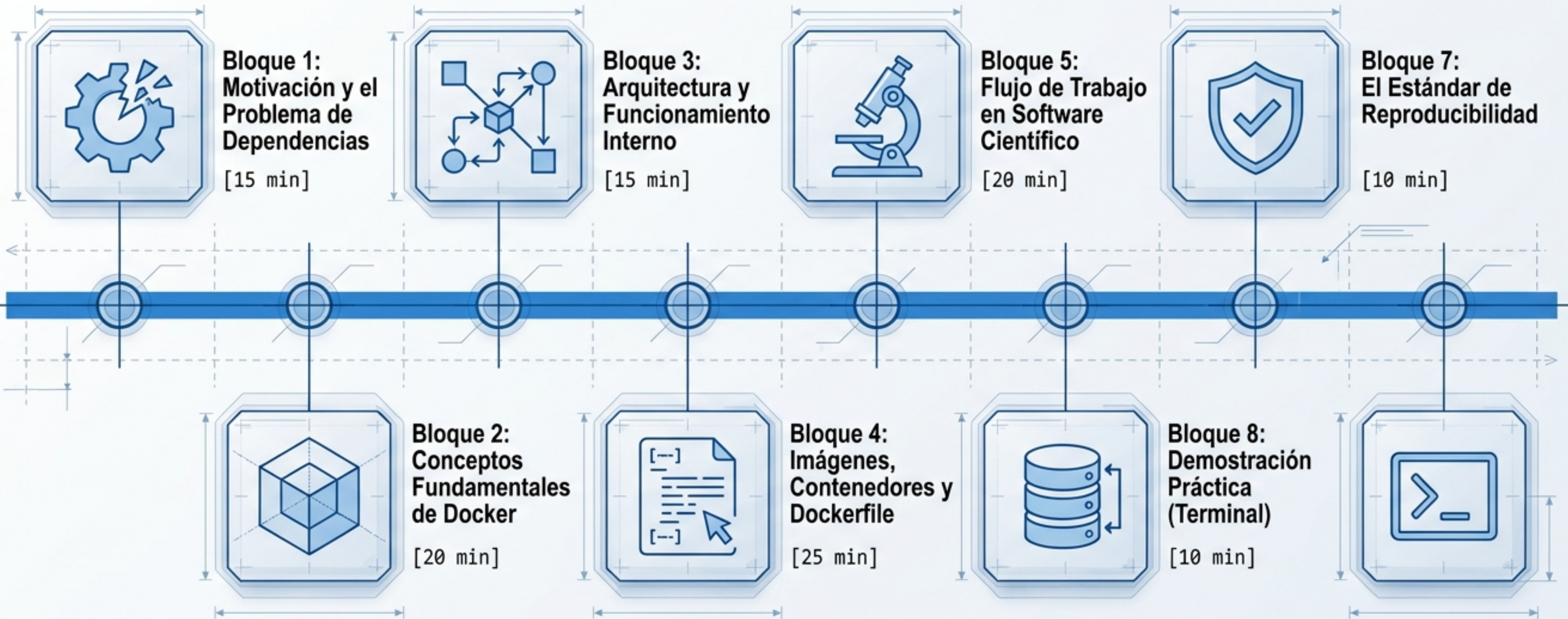
Curso: Desarrollo de Software Científico

Módulo: Machine Learning Operations (MLOps)

Duración Estimada: 120 minutos



Hoja de Ruta del Laboratorio (120 Minutos)

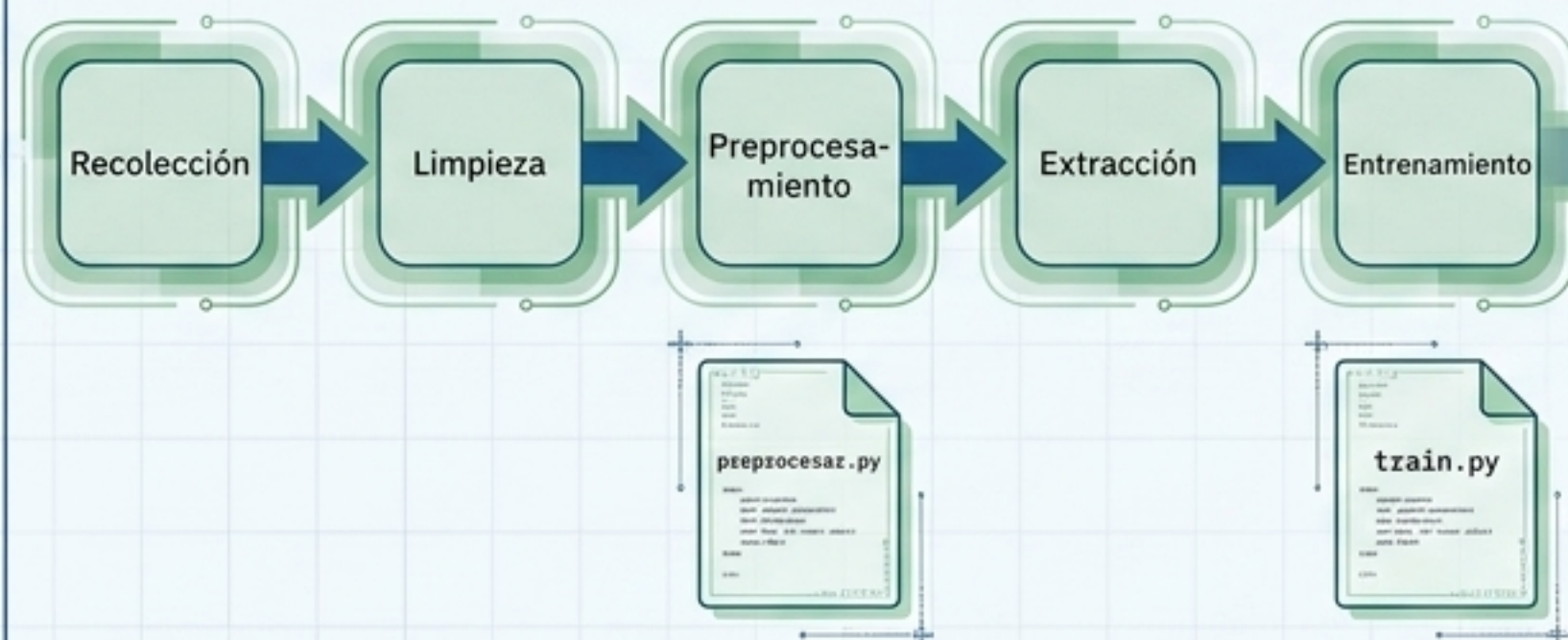


La Paradoja del Desarrollo Científico

Un código no es solo lo que escribimos, es el entorno en el que vive.

16/9

Fase 1: Prototipo Local (Laptop)



Fase 2: Producción (Servidor Cloud)



Éxito local comprobado.

Diagnóstico Clínico: La Anatomía de una Dependencia Rota

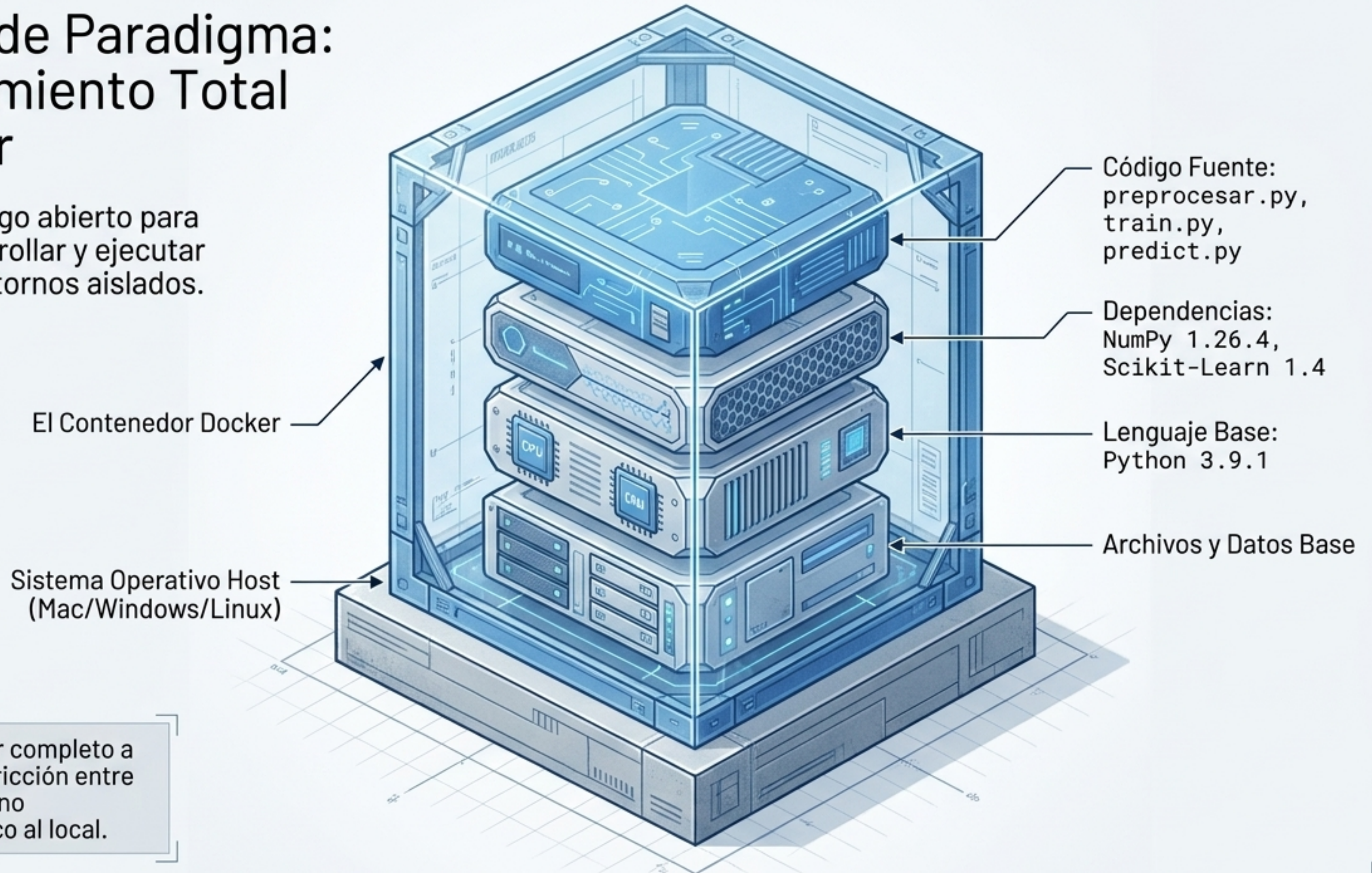
Desplegar únicamente el código fuente garantiza el fracaso del proyecto en entornos remotos.

Componente	Entorno Local (Laptop)	Entorno Remoto (Servidor)	Estado de Compatibilidad
Scripts Base	train.py, predict.py	train.py, predict.py	✓ Idéntico
Lenguaje Core	Python 3.9.1	Python 3.8.4	✗ Conflicto
Cálculo Numérico	NumPy 1.26.4	NumPy 1.23.2	✗ Conflicto
Machine Learning	Scikit-Learn 1.4	Scikit-Learn 1.3	✗ Conflicto

El problema de 'Funciona en mi máquina': Las dependencias implícitas (SO, versiones de lenguaje y librerías) no viajan con los scripts.

El Cambio de Paradigma: Encapsulamiento Total con Docker

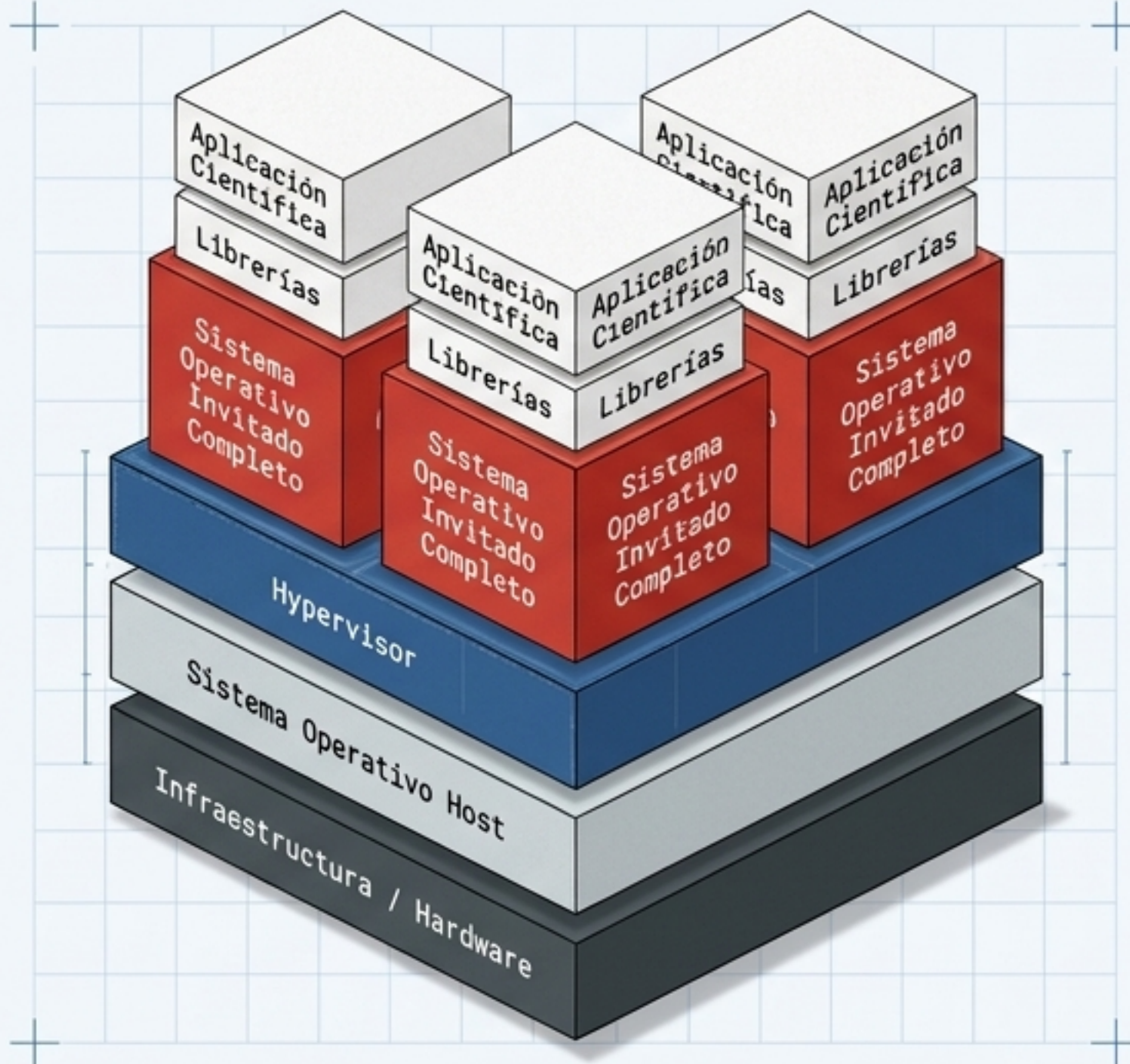
Plataforma de código abierto para empaquetar, desarrollar y ejecutar aplicaciones en entornos aislados.



Eficiencia Computacional: Contenedores vs. Máquinas Virtuales

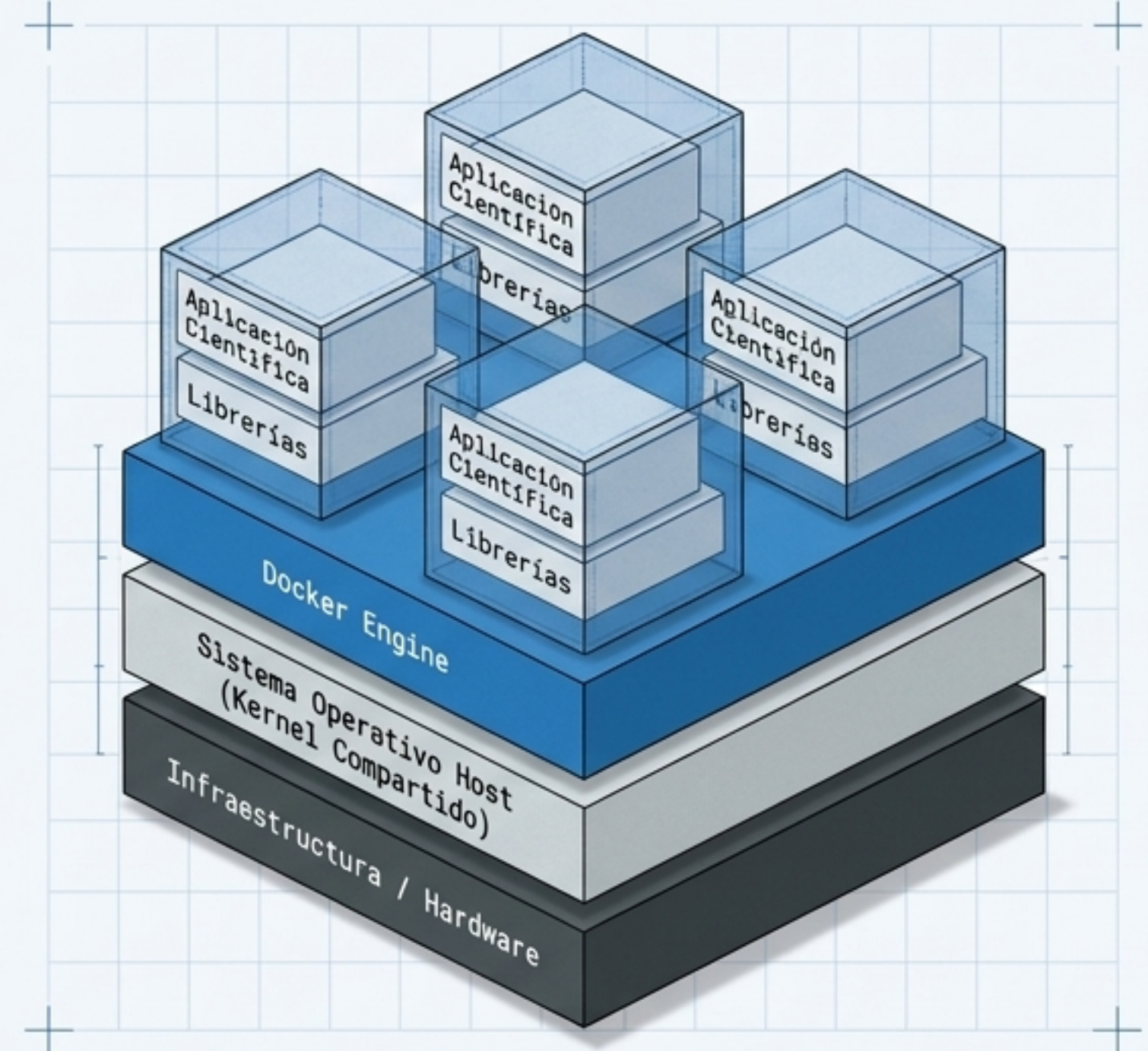
Por qué el modelado científico requiere arquitecturas ligeras.

Máquina Virtual (VM) - Arquitectura Pesada



Requiere duplicar un sistema operativo por cada proyecto, consumiendo gigabytes de RAM.

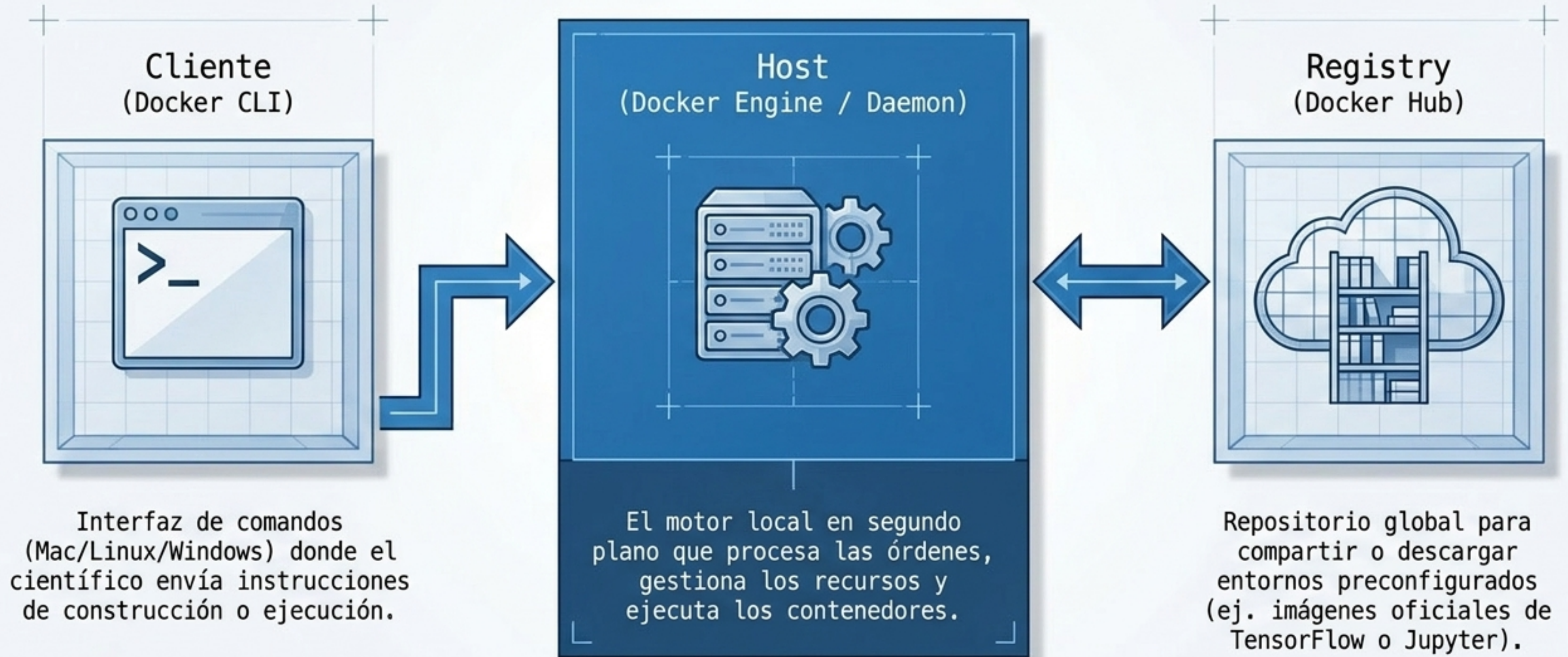
Contenedor (Docker) - Arquitectura Ligera



Comparte el núcleo del Host. Arranque en milisegundos, ideal para despliegues masivos.

Arquitectura Interna: El Motor de Docker

Un modelo Cliente-Servidor diseñado para la distribución global.



El Ciclo de Vida del Software Científico en Docker

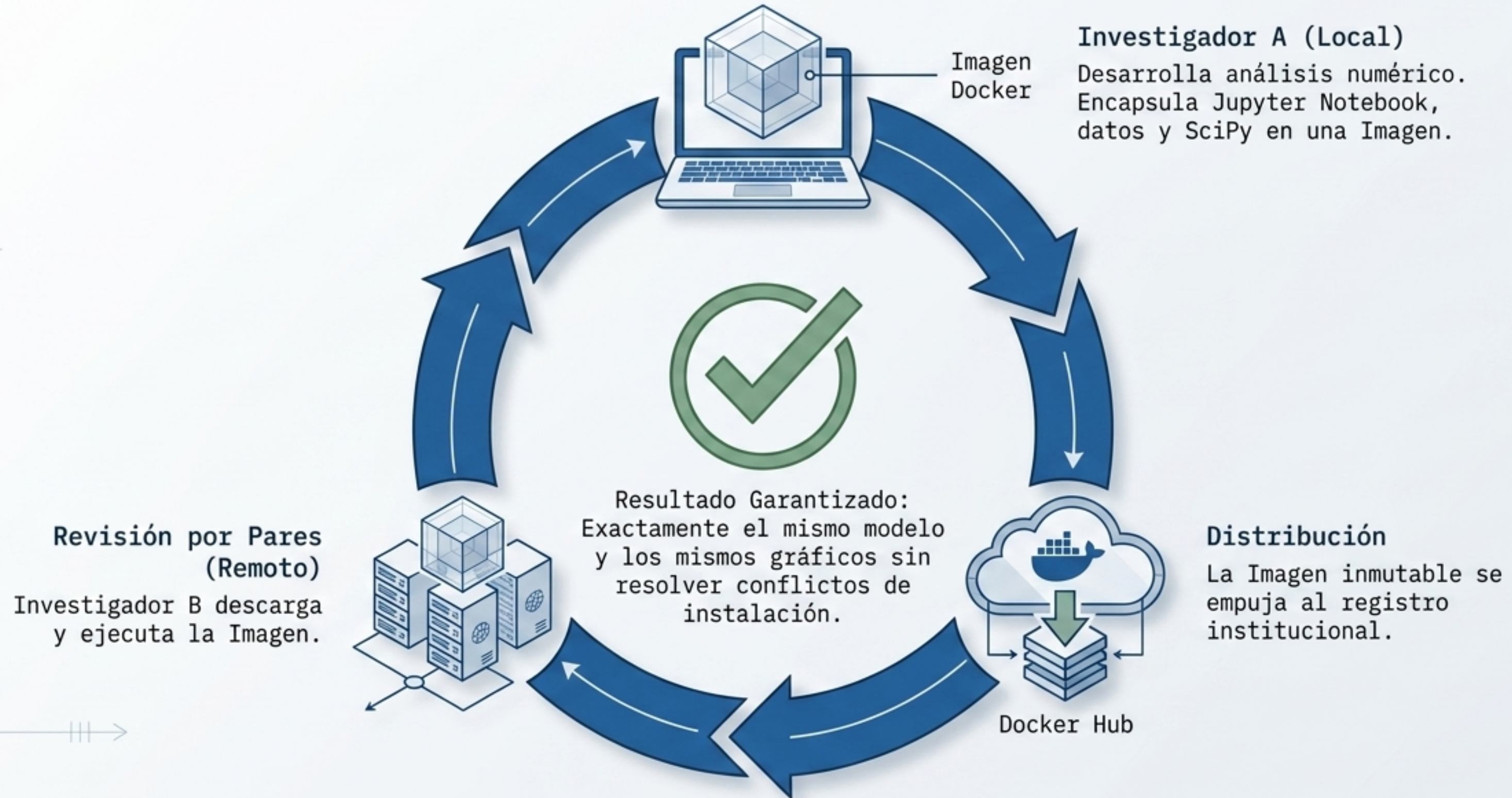
Del plano arquitectónico a la ejecución viva.

////// The Digital Laboratory Environment



Solución a la Crisis de Reproducibilidad

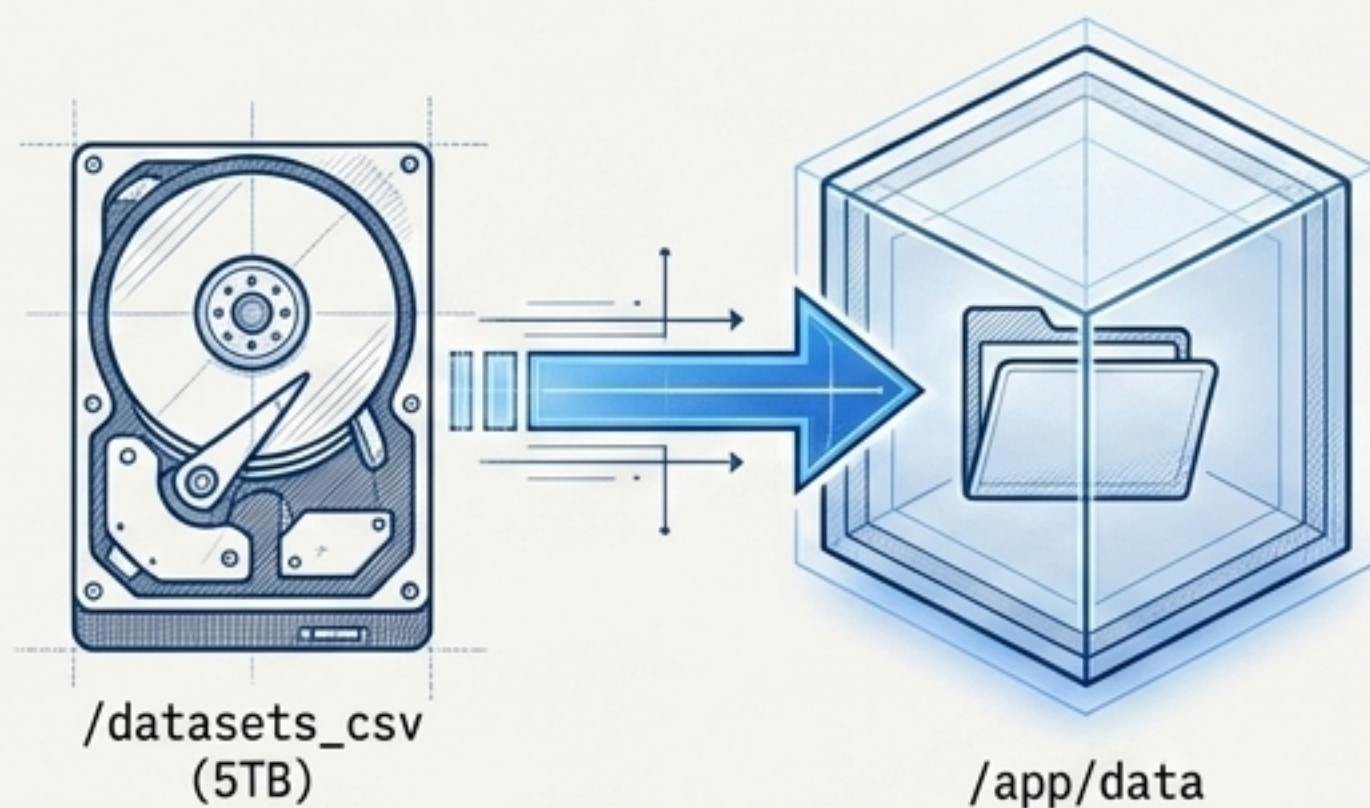
Garantizando exactitud numérica en la colaboración global.



Más Allá del Aislamiento: Persistencia y Composición

Manejo de conjuntos de datos masivos y arquitecturas complejas.

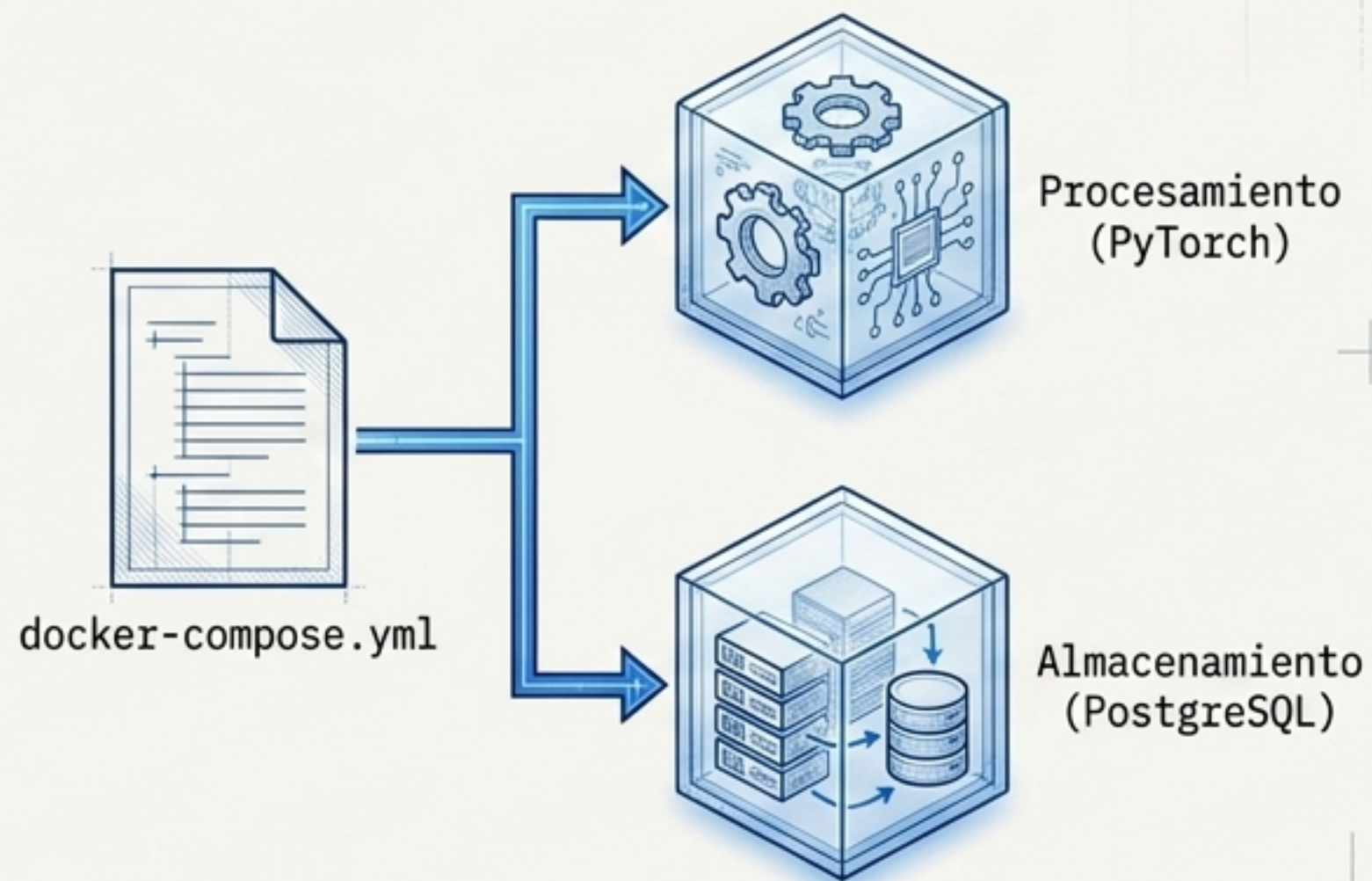
Volúmenes (Persistencia de Datos)



Problema: Los contenedores son efímeros.

Solución: Un puente lee terabytes de datos externos sin inflar el tamaño de la imagen original.

Docker Compose (Orquestación Científica)



Problema: Simulaciones con múltiples servicios interactuando.

Solución: Un archivo de configuración que levanta un clúster científico entero con un solo comando.

Código Limpio, Contenedores Ligeros

Decálogo de buenas prácticas para el científico de datos.



Fijar versiones siempre

Especificar librerías exactas (ej. `scikit-learn==1.4`) para asegurar la reproducibilidad.



Evitar etiquetas genéricas

Nunca usar la etiqueta `:latest` en entornos científicos; rompe la reproducibilidad con el tiempo.



Imágenes base optimizadas

Partir de versiones ligeras (ej. `python:3.9.1-slim`) en lugar de SO completos para acelerar despliegues.



Ignorar ruido local

Utilizar `.dockerignore` para excluir entornos virtuales (`.venv`) o datasets locales masivos.



Un contenedor = Un propósito

Separar físicamente la recolección de datos del entrenamiento del modelo.

Práctica de Laboratorio: Instalación y Verificación Local

Preparando la infraestructura para construir nuestra primera imagen.

Proceso de Adquisición

Paso 1: Visitar `docker.com` -> 'Get Started'.

Paso 2: Descargar ejecutable (.exe / .dmg).
La capa gratuita es suficiente para el desarrollo académico.

Paso 3: Instalación estándar (requiere privilegios de administrador). Iniciar sesión es opcional.

Si la terminal responde con el listado adjunto, el motor está funcionando. Si existe un error, capture la pantalla para diagnóstico inmediato.

```
$ docker
```

```
Usage:  docker [OPTIONS] COMMAND
```

```
A self-sufficient runtime for containers
```

```
Commands:
```

```
  build  Build an image from a Dockerfile
```

```
  run    Run a command in a new container
```

```
  ...
```